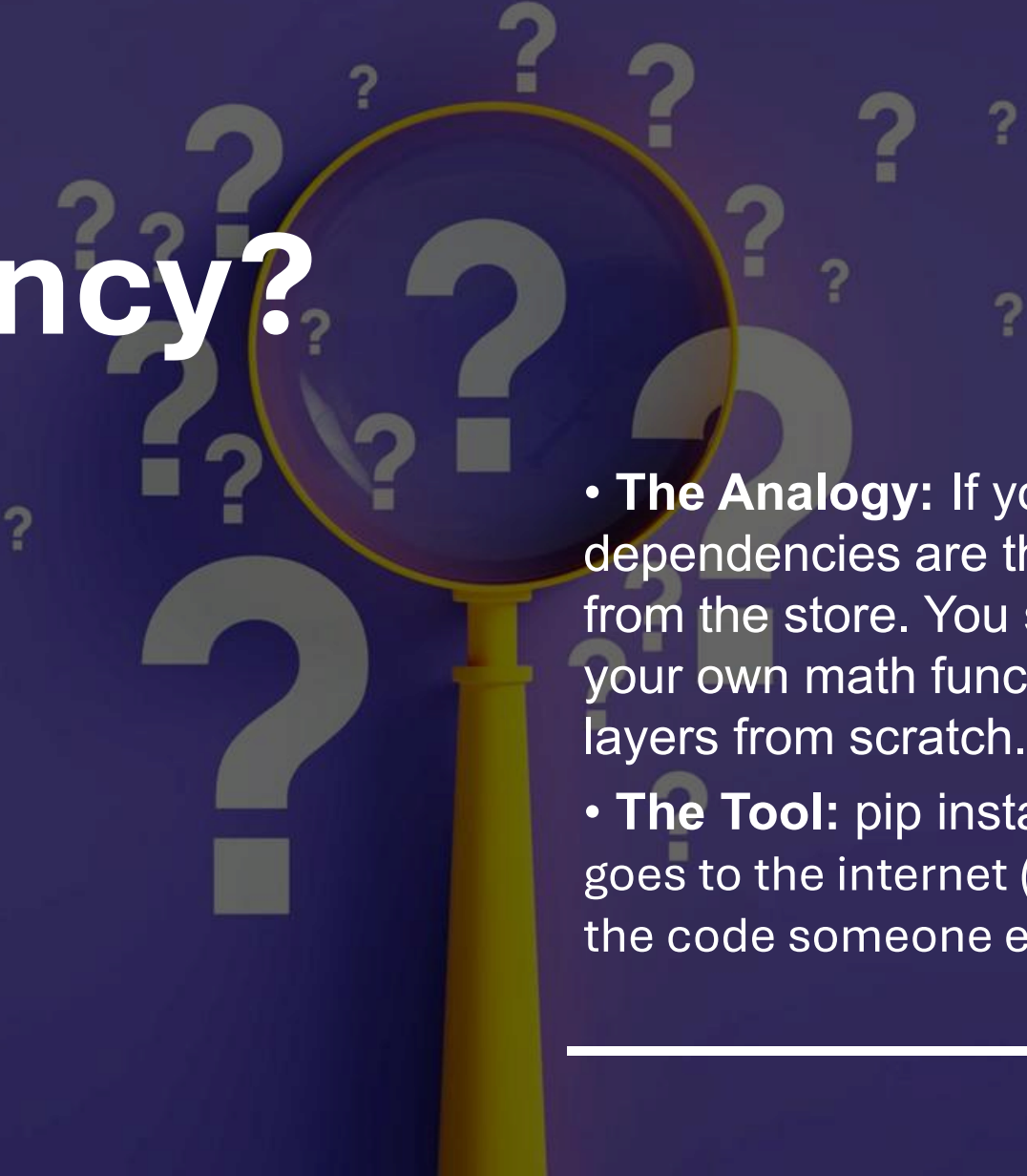

What is a Dependency?

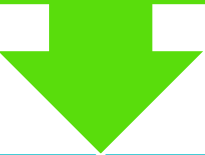
- 
- **The Analogy:** If your code is a recipe, dependencies are the ingredients you buy from the store. You shouldn't have to write your own math functions or neural network layers from scratch.
 - **The Tool:** `pip install <package>`. This goes to the internet (PyPI) and downloads the code someone else wrote.
-

The Basic Standard: requirements.txt

The Problem: If you share your project, the other person needs exactly the same ingredients.



The Solution: A simple text file listing all libraries.



Pinning: Explain the difference between `pip install numpy` (gets whatever the newest version is today) and `pip install numpy==1.24.3` (gets the exact version you tested your code with).

The Advanced Tier: The Lockfile (Poetry / Pipenv)

The "Dependency Hell" Problem:

Even if you pin Library A, it might secretly install Library B in the background. If Library B updates, your code breaks silently. This is fatal when relying on specific CUDA toolkits or precise versions of PyTorch for complex computer vision models.

The Solution: Lockfiles (poetry.lock or Pipfile.lock). These tools calculate a massive tree of *every single sub-dependency* and lock them to an exact cryptographic hash. This guarantees the environment builds perfectly on any machine, every single time.

The Pro-Tip: Caching & Mirroring

The Challenge: Downloading massive 2GB AI frameworks repeatedly wastes time, especially in lab environments where internet speeds can fluctuate or drop.

The Technique: Configuring local package caches and offline wheels. This allows a team to download heavy libraries once and install them locally across multiple machines without needing an active connection.